CS 559: REAL-TIME MARKET DATA FORECASTING

Syed Yasir ALam
Stevens Institute of Technology
salam4@stevens.edu
Fall 2024

ABSTRACT

This course project focuses on building predictive models for real-world financial market data, as part of the Jane Street trading challenge. The problem involves modeling `responder_6`, a key variable in a dataset derived from production systems, using anonymized and obfuscated features. The challenge highlights unique difficulties in financial modeling, including non-stationary time series, fat-tailed distributions, and the dynamic nature of human-influenced markets.

Successfully addressing this problem can demonstrate the applicability of machine learning to quantitative trading, with potential implications for improving automated trading strategies in highly complex and competitive environments. The evaluation metric for this project is the sample-weighted zero-mean R^2 (R-squared) score, ensuring a rigorous assessment of model performance. Preliminary results show that thoughtful feature engineering and robust modeling techniques can mitigate some challenges posed by the data, offering promising avenues for further refinement and generalization.

1 Introduction

Modeling financial markets presents a unique and formidable challenge due to the inherent complexities of market behavior, such as non-stationary time series, fat-tailed distributions, and sudden regime shifts. These characteristics make traditional statistical approaches insufficient and necessitate innovative techniques to understand and predict market dynamics. The importance of addressing this problem cannot be overstated, as effective predictive models can improve decision-making processes, optimize trading strategies, and enhance market efficiency.

This project is motivated by the Jane Street trading challenge, which involves developing a model to predict `responder_6`, a critical variable in a dataset derived from real-world production systems. The dataset reflects actual trading environments where automated strategies are employed, providing an authentic glimpse into the challenges faced in quantitative finance. The anonymization and obfuscation of features in the dataset ensure the problem's difficulty while maintaining the relevance of the underlying task.

The objective of this project is to explore and implement machine learning techniques to build a robust predictive model for `responder_6`, evaluated using the sample-weighted zero-mean R^2 (R-squared) score. By tackling this problem, the project aims to bridge theoretical approaches with practical applications in trading, providing insights into how machine learning can navigate the uncertainties of financial markets. Through this work, we seek to contribute to the understanding of how modern predictive models can be leveraged in highly competitive and dynamic trading environments.

2 Related Work

Predictive modeling in financial markets has been a longstanding topic of interest, with researchers and practitioners exploring various approaches to tackle the unique challenges posed by these data. Traditional methods often rely on statistical models such as autoregressive integrated moving average (ARIMA) and generalized autoregressive conditional heteroskedasticity (GARCH) to handle time series forecasting. While these methods are effective in stable environments, their assumptions of stationarity and linearity limit their applicability in dynamic market scenarios characterized by regime shifts and complex dependencies.

In recent years, machine learning (ML) approaches have gained prominence in financial modeling due to their ability to capture non-linear relationships and adapt to non-stationary data. Techniques such as gradient-boosted decision trees (e.g., XGBoost, LightGBM) and deep learning models, including long short-term memory networks (LSTMs) and transformers, have been applied to various financial tasks like price prediction, risk assessment, and portfolio optimization. Notable works have demonstrated the potential of ensemble methods and recurrent neural networks to outperform traditional models in specific market conditions.

However, these ML approaches also face limitations. A significant challenge is the lack of interpretability, which is critical for decision-making in high-stakes financial applications. Additionally, financial markets are influenced by factors beyond the scope of historical data, such as geopolitical events and technological advancements, making models prone to overfitting and underperformance in unseen scenarios. Moreover, obtaining high-quality labeled datasets is challenging due to proprietary restrictions, leading to reliance on heavily anonymized or obfuscated data that may obscure important features.

This project builds on prior work by addressing the challenges of non-stationarity, feature obfuscation, and interpretability. By leveraging the anonymized dataset provided in the Jane Street trading challenge, we aim to develop a model that balances accuracy with robustness, contributing to the broader understanding of how machine learning can enhance quantitative trading strategies in complex financial environments.

3 Methodology

3.1 Problem Formulation

The task is to build a predictive model for `responder_6`, a continuous target variable in an anonymized financial time series dataset derived from Jane Street's production systems. Given a dataset with n samples and m features, let:

- $\mathbf{X} \in \mathbb{R}^{n \times m}$ denote the feature matrix, where each row represents a sample and each column represents an anonymized feature.
- $\mathbf{y} \in \mathbb{R}^n$ represent the ground-truth values for `responder_6`, clipped between -5 and 5.
- $\hat{\mathbf{y}} \in \mathbb{R}^n$ denote the predicted values for `responder_6`.
- $\mathbf{w} \in \mathbb{R}^n$ denote the sample weights associated with each observation.

The competition evaluates models using the sample-weighted zero-mean R^2 (R-squared) score:

$$R^2 = 1 - \frac{\sum_{i=1}^{n} w_i (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} w_i y_i^2}.$$

The primary objective is to maximize R^2 by minimizing the weighted mean squared error (WMSE) in the numerator.

3.2 Dataset Description

The competition dataset consists of a time-series dataset containing **79 features** and **9 responders**, anonymized but derived from real market data. The main objective of this competition is to forecast **responder_6** for up to six months into the future.

Dataset Structure

The dataset is provided in the following files:

- **train.parquet**: The training dataset containing historical data and returns. For convenience, the dataset is partitioned into 10 parts.
- **test.parquet**: A mock test set that represents the structure of the unseen test set. This set demonstrates a single batch served by the evaluation API.
- **lags.parquet**: Contains lagged values of responder_0 to responder_8 by one date_id.
- **sample_submission.csv**: Demonstrates the required format for model predictions.
- **features.csv**: Metadata related to the anonymized features.
- **responders.csv**: Metadata related to the anonymized responders.

Each row in the `train.parquet` and `test.parquet` files corresponds to a unique combination of:

- **symbol_id**: Encrypted identifier for a financial instrument.
- **date_id**: An integer representing the event day.
- **time_id**: An integer providing time ordering within a day.

It is important to note that:

- The `time_id` values are ordinally sorted, but the actual time intervals between consecutive values may vary.
- Each `symbol_id` does not necessarily appear in all combinations of `date_id` and `time_id`.
- New `symbol_id` values may appear in future test sets.

Target Variable

The target variable for this competition is **responder_6**, which is one of the 9 responders (`responder_0` to `responder_8`) provided in the dataset. The responders are clipped within the range $[-5, 5]$.

Field Information

The key columns in the dataset include:

- **date_id** and **time_id**: Ordinal integer values for chronological order.
- **symbol_id**: Unique identifier for financial instruments.
- **weight**: Used for calculating the scoring function.
- **feature_00 to feature_78**: Anonymized market data features.
- **responder_0 to responder_8**: Target responders, with **responder_6** as the competition target.
- **is_scored**: Indicates whether a row contributes to the evaluation metric.

The provided lagged responders (`lags.parquet`) include all responders from the previous `date_id` at the first `time_id` of the subsequent day.

Remarks on Time-Series Data

The structure of the dataset reflects a real-world financial market setting. It presents several challenges, including:

- Irregular time intervals between consecutive time_id values.
- The potential introduction of new symbol_id values in unseen test sets.
- The need for sequential predictions during the forecasting phase.

These challenges require robust and adaptive models capable of handling dynamic financial data.

3.3 Data Preprocessing

Due to the high volume of data, manual feature engineering was not performed. Instead, the focus was on leveraging the model's capacity to learn meaningful features automatically. In certain experiments, the input features were mapped to a much higher-dimensional space and subsequently back to a lower-dimensional representation using linear layers implemented in PyTorch. This allowed the models to learn complex feature interactions while discarding noise.

The preprocessing steps included the following:

Feature Scaling

- Tests were conducted both with and without normalization. For experiments requiring scaling, standardization (z-scores) was applied to ensure uniformity across predictors.

Handling Missing Data

- Missing values were addressed using median imputation to maintain robustness against outliers.

Data Organization and Sequence Generation

- The data was sorted chronologically using `date_id` and `time_id`.
- There were 39 symbols, each represented by 79 anonymized features. At every time step, predictions were required for all 39 symbols.
- For LSTM-based models, sequences were generated using a sliding window approach. Care was taken to ensure that sequences did not mix multiple symbols within the same input, preserving the temporal consistency of each symbol's data.
- During validation, the dataset was transformed into batches where each batch contained 39 symbols corresponding to a particular `date_id` and `time_id`.

4 Modeling Approach

The predictive modeling approach involved training and evaluating a variety of machine learning techniques, including feedforward neural networks, LSTMs, and gradient-boosted decision trees. Special consideration was given to methods that could handle the temporal and structural complexities of the dataset.

The following models were implemented and evaluated:

1. Simple MLP (Multi-Layer Perceptron)

- A feedforward neural network with two hidden layers was implemented as a baseline.
- ReLU activation was applied to the hidden layers, and the output layer used linear activation for regression.

2. MLP with Symbol Embedding

- An embedding layer was introduced to allow the model to adapt predictions for each symbol.
- The embeddings enabled the model to capture symbol-specific patterns and improve performance across symbols.

3. Symbol-Specific Models

- Separate models were trained for each of the 39 symbols.
- During evaluation, predictions for each symbol were made using its corresponding model. This approach allowed the models to specialize in the dynamics of individual symbols.

4. LSTM with Single Layer

- A single-layer LSTM model was implemented to capture temporal dependencies within the sequences.
- Input sequences were constructed for each symbol using a sliding window, ensuring temporal consistency.

5. LSTM with Symbol Embedding

- Similar to the MLP with symbol embedding, an embedding layer was added to the LSTM to enable symbol-specific adjustments in predictions.
- The combined LSTM and embedding architecture allowed the model to capture both temporal and symbol-specific patterns.

6. LSTM Experts

- Multiple LSTM expert models were trained, each specializing in specific subsets of the data.
- During inference, the predictions from the experts were combined to make the final prediction.

7. XGBoost Algorithm

- Gradient-boosted decision trees (XGBoost) were employed as a non-neural baseline to capture non-linear relationships in the data.
- Hyperparameters, such as the learning rate, maximum tree depth, and number of estimators, were tuned using Bayesian optimization.

Validation and Evaluation

- A time-based split was used for cross-validation to ensure that the validation data was always forward-looking and did not leak information from the future.
- During validation, the dataset was organized into batches where each batch contained the 39 symbols corresponding to a specific `date_id` and `time_id`.
- The primary evaluation metric was the sample-weighted zero-mean R^2 score to align with the competition's scoring function.

Online Training Pipeline

To address the significant risk of distribution shift observed in the dataset, an online training pipeline was developed. This allowed the models to adapt dynamically to incoming data. Specifically:

- The model was updated on-the-fly with the most recent observations.
- This approach ensured that the models remained robust to changes in the underlying data distribution and maintained performance over time.

4.1 Pipeline Overview

The complete methodology can be summarized as follows:

1. Data preprocessing: Scaling, handling missing values, and sequence generation.
2. Model training: Implementation of multiple architectures (MLP, LSTM, XGBoost).
3. Hyperparameter tuning: Optimization using Bayesian methods.
4. Cross-validation: Time-based splits ensuring temporal consistency.
5. Online training: Real-time model adaptation to counter distribution shifts.
6. Model evaluation: Computation of the sample-weighted R^2 score.

This systematic and expansive approach ensured that all aspects of the dataset were explored, and no potential modeling avenue was left untested. The models were optimized to balance simplicity, interpretability, and robustness while addressing the unique challenges of the competition dataset.

4.2 Loss Function Evaluation

Several loss functions were evaluated to identify the most suitable approach for optimizing the model's performance, maximizing R^2, and ensuring robustness to data shifts.

1. Mean Squared Error (MSE)

- **Reasoning:** MSE is the most common loss for regression tasks. It penalizes large errors more than small ones, making it sensitive to outliers. This sensitivity makes it a good initial candidate for models that must capture the majority of data variance.
- **Intended Problem to Solve:** It minimizes overall squared error and aligns closely with maximizing R^2 as both focus on variance reduction.

2. Weighted Mean Squared Error (WMSE)

- **Reasoning:** Incorporating sample weights ensures that predictions prioritize critical observations (e.g., large values, high confidence points).
- **Intended Problem to Solve:** The dataset may have uneven importance across observations, so WMSE prevents underfitting or misweighting of significant samples.

3. Mean Absolute Error (MAE)

- **Reasoning:** MAE is less sensitive to outliers compared to MSE as it applies linear error penalties. This property is useful for datasets with noisy or skewed data.
- **Intended Problem to Solve:** Reduces the risk of overfitting to outliers and provides a more robust error measure when the data distribution contains anomalies.

4. Weighted Mean Absolute Error (WMAE)

- **Reasoning:** WMAE combines the robustness of MAE with sample-specific importance weighting. It is particularly useful for imbalanced datasets.
- **Intended Problem to Solve:** Ensures that both robustness (to outliers) and sample-specific priorities are maintained during training.

5. Huber Loss

- **Reasoning:** Huber loss combines the benefits of MSE (quadratic penalties for small errors) and MAE (linear penalties for large errors). This makes it robust to outliers while maintaining sensitivity to small deviations.
- **Intended Problem to Solve:** Provides a balanced loss function for datasets prone to both large errors and subtle shifts, ensuring robustness and stability during training.

Evaluation Results

Each loss function was evaluated based on its ability to:

- Optimize the R^2 score.
- Remain robust to data shifts observed over time.
- Handle the presence of outliers and noisy observations effectively.

5 Results

5.1 Loss Function Evaluation

To identify the most suitable loss function for training, we evaluated Mean Squared Error (MSE), Weighted MSE, Mean Absolute Error (MAE), Weighted MAE, Huber Loss, and Weighted Huber Loss. Table 1 summarizes their respective performances based on the sample-weighted zero-mean R^2 score.

From the results, **Weighted MSE** achieved the best R^2 score of **-0.007**, demonstrating its effectiveness in incorporating sample-specific weights to prioritize critical observations. The Weighted MAE also performed relatively well, highlighting the robustness of absolute error-based metrics in noisy environments.

5.2 Model Architecture Evaluation

We tested multiple model architectures, including Multi-Layer Perceptrons (MLP), LSTM-based models, and XGBoost. Each architecture was evaluated across three conditions: standard evaluation, online training adaptation, and normalization. Table 2 provides the detailed R^2 scores achieved for each experiment.

The **LSTM Experts** model achieved the highest R^2 score of **0.507** under online training conditions, demonstrating its capability to handle temporal dependencies and adapt dynamically to evolving data. Additionally, normalization significantly improved the performance of all models, with LSTM-based architectures showing consistent gains when embedding layers and expert models were introduced.

These results indicate that the **Weighted MSE loss** combined with LSTM-based architectures, particularly expert models, delivers the most robust performance for predicting the target variable, responder 6. Further experiments incorporating online training pipelines and normalization provide promising avenues for addressing distribution shifts in real-world financial markets.

References

@articlehuber1964loss, title=Robust estimation of a location parameter, author=Huber, Peter J., journal=The Annals of Mathematical Statistics, volume=35, number=1, pages=73–101, year=1964

@articlehochreiter1997lstm, title=Long short-term memory, author=Hochreiter, Sepp and Schmidhuber, Jürgen, journal=Neural computation, volume=9, number=8, pages=1735–1780, year=1997, publisher=MIT Press

@articlekingma2014adam, title=Adam: A method for stochastic optimization, author=Kingma, Diederik P and Ba, Jimmy, journal=arXiv preprint arXiv:1412.6980, year=2014

@articlechen2016xgboost, title=XGBoost: A scalable tree boosting system, author=Chen, Tianqi and Guestrin, Carlos, booktitle=Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages=785–794, year=2016, organization=ACM

@inproceedingsgraves2013generating, title=Generating sequences with recurrent neural networks, author=Graves, Alex, booktitle=arXiv preprint arXiv:1308.0850, year=2013

@inproceedingssutskever2014sequence, title=Sequence to sequence learning with neural networks, author=Sutskever, Ilya and Vinyals, Oriol and Le, Quoc V, booktitle=Advances in neural information processing systems, pages=3104–3112, year=2014

Table 1: Testing Different Losses during Training

Loss Methods	**R Squared Score**
Mean Squared Error	-0.12
Weighted MSE	-0.007
Mean Absolute Error	-0.35
Weighted MAE	-0.28
Huber Loss	-0.51
Weighted Huber Loss	-0.49

Table 2: Evaluating Different Model Architectures

Model Architecture	**Evaluation**	**Online Training**	**Normalization**
MLP	-0.007	0.017	0.065
MLP with embedded layers	0.0003	0.076	0.114
MLP Experts	0.0002	0.126	0.329
LSTM	-0.004	0.113	0.235
LSTM with embedded layers	0.002	0.214	0.323
LSTM Experts	0.026	0.384	0.507
LSTM with experts and clipped outputs	-	-	0.493
XGBoost	0.00843	-0.00719	-